Cocoon control flow

Ovidiu Predescu

November 18, 2003

About the author

- Writes software for fun and for a living
- Involved with many open-source projects: GNUstep, GCC, XEmacs, Apache Cocoon etc.
- Current employer: Google

What is Cocoon?

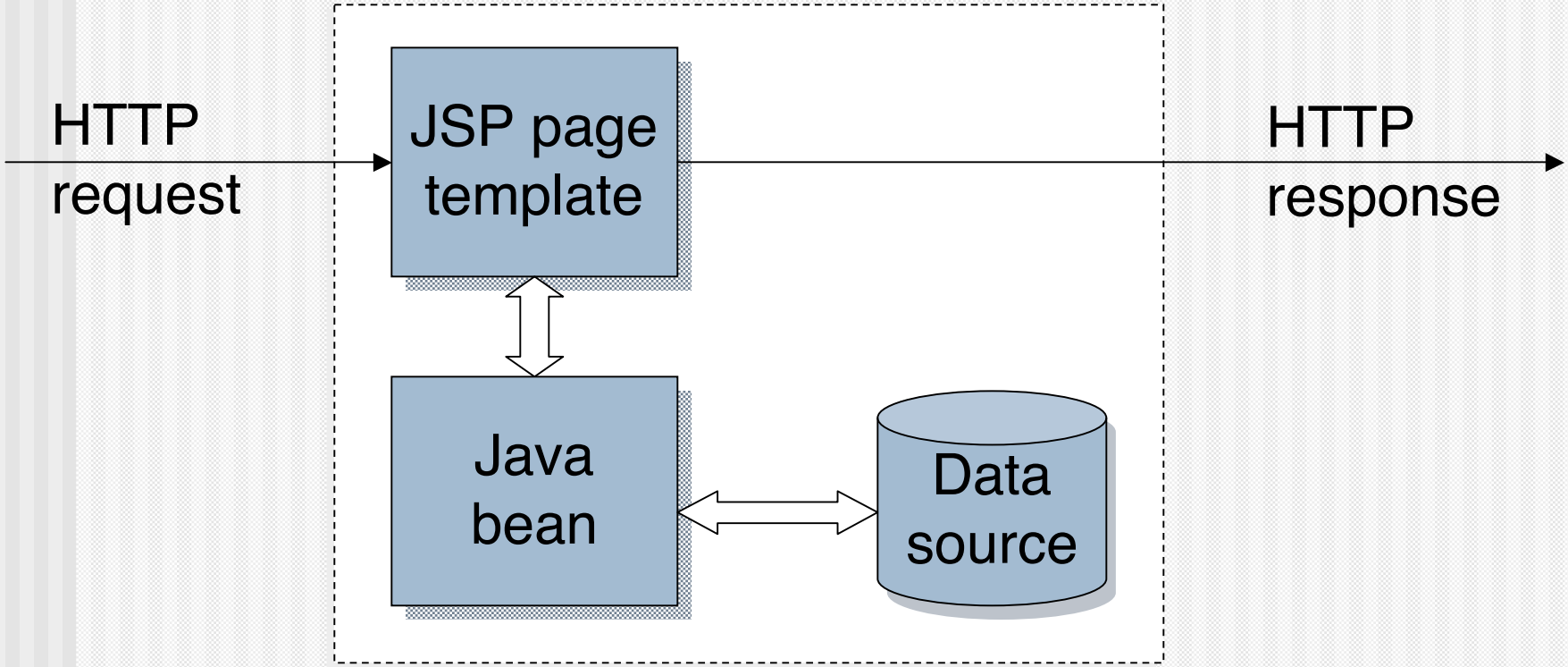
- Powerful XML processing engine.
- Neat features for writing Web applications.

Web app frameworks today

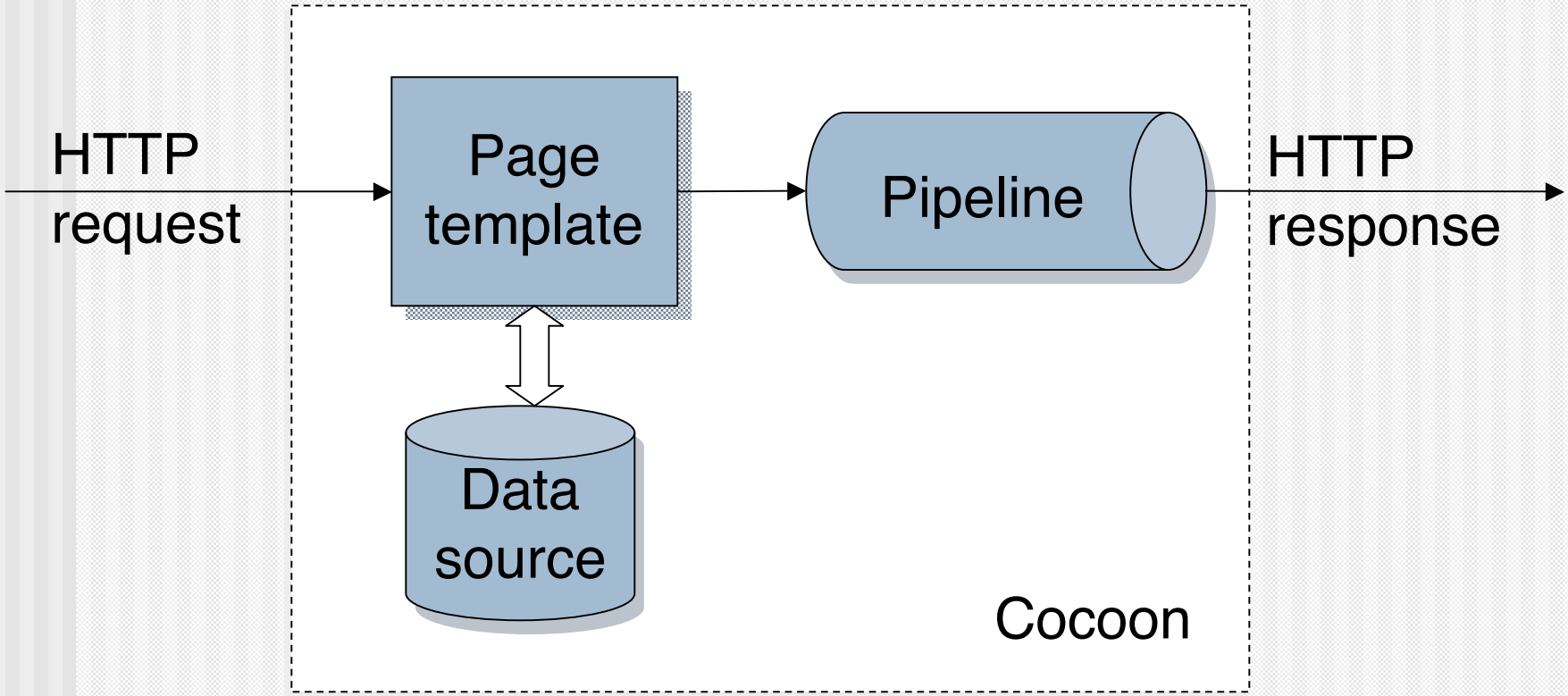
- “Model 1”:
  - page oriented (logic is placed in the page template)
- “Model 2”
  - Model-View-Controller (logic and page template are separated)

Source http://www.javaworld.net/javaworld/jw-12-1999/jw-12-ssj-jspmvc.html

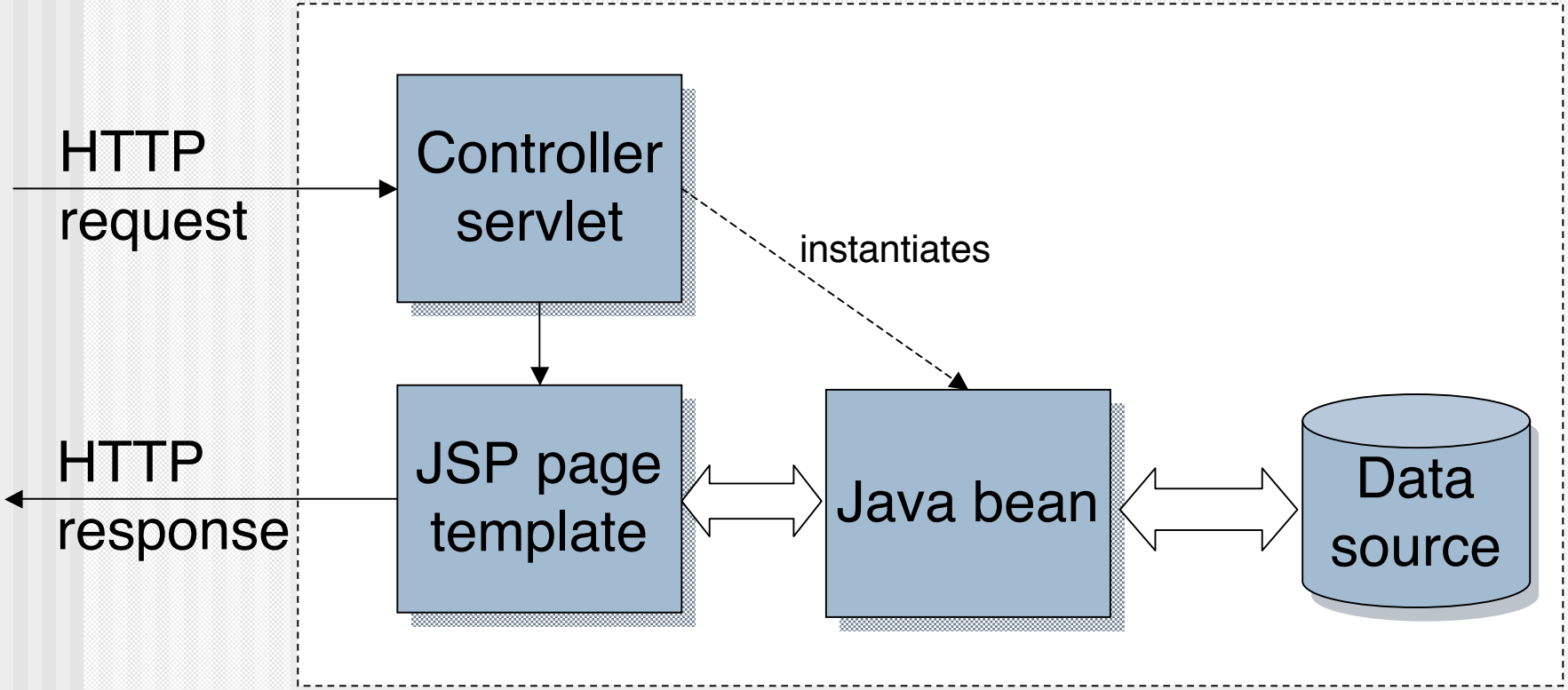
“Model 1” in the servlet world



“Model 1” in Cocoon



“Model 2” in the servlet world

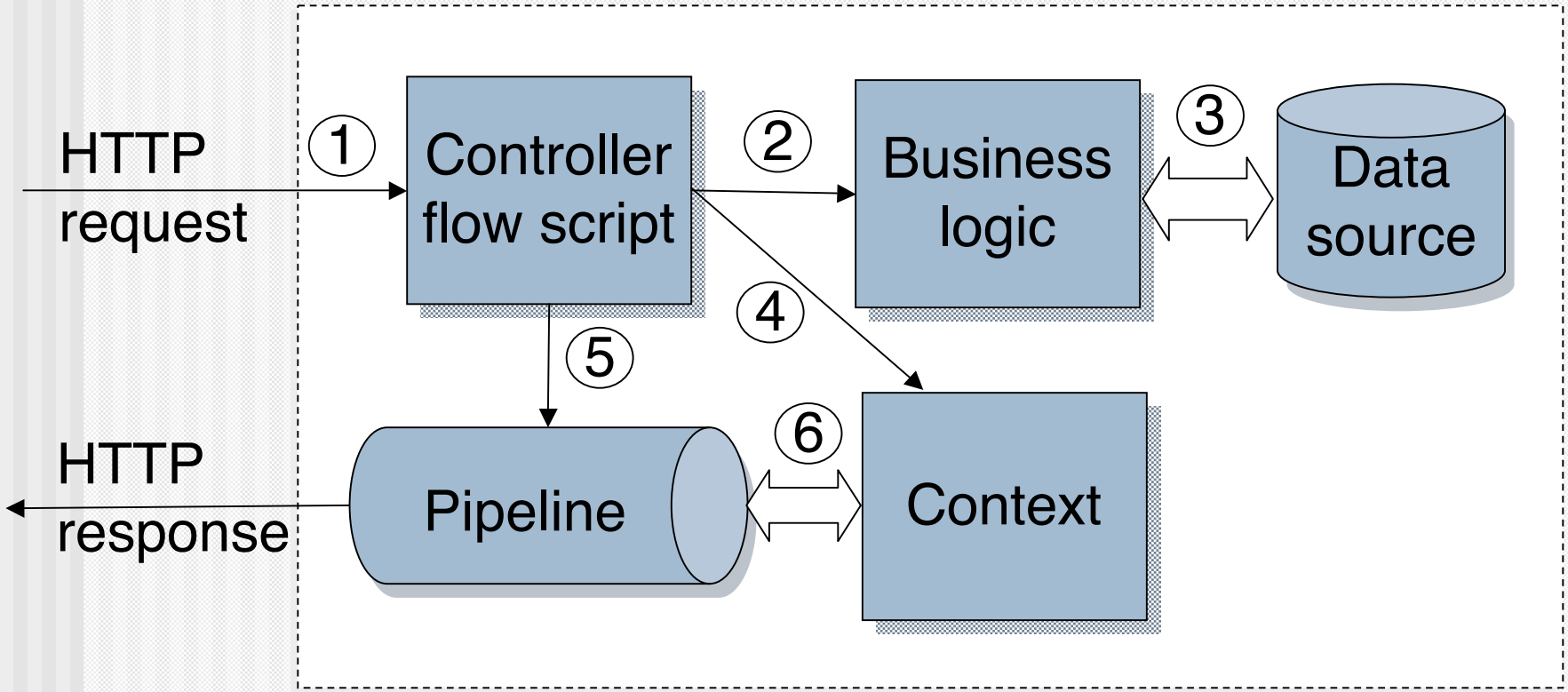


Benefits of “Model 2”

- Logic and controlling the flow of pages are separated from presentation
- Uses the well-understood Model-View-Controller pattern
- Easier to scale as the application (read number of pages) grows

What is the “Model 2” equivalent in Cocoon?

Control flow: MVC+ for Cocoon



The Controller layer

- Also referred to as “control flow layer”
- Simple and effective way to glue together business logic, presentation and page flow
- Use scripts written in JavaScript, a simple scripting language, to implement the most complex use cases

The Controller layer (cont)

- Uses a modified version of the Rhino JavaScript engine that supports continuations (developed by Christopher Oliver)
- Easy to add support for languages with first-class continuations: Scheme, even Java (Brakes project, ATCT)

Sample flow script

```
function checkout()
{
  var user, cart;
  while (user == null) {
    sendPageAndWait("login.xml");
    user = UserRegistry.getUser(cocoon.request.get("name"))
  }
  sendPageAndWait("shippingAddress.xml");
  var address = cocoon.request.get("address");
  sendPageAndWait("creditCard.xml");
  var creditCard = cocoon.request.get("creditCard");
  sendPageAndWait("placeOrder.xml");
  EnterpriseSystem.placeOrder(cart);
  sendPage("orderPlaced.xml");
}
```

JavaScript vs. Java detour

- If you know Java, you already know JavaScript! Well, mostly.
- Dynamically typed (variables don't have types, values do)
- Prototype-based inheritance as opposed to class-based inheritance
- Objects are extensible at runtime: add or remove properties and methods

`sendPageAndWait` special function

- `sendPageAndWait` passes the control to sitemap to generate the output page (View)
- `sendPageAndWait` takes two arguments:
  - an URL relative to current sitemap
  - context object to be made available to the output page template
- The flow script is suspended after View is generated, and its whole execution stack saved in a **continuation** object

Flow script revisited

saved continuations

```
function checkout()
{
  var user, cart;
  while (user == null) {
    sendPageAndWait("login.xml");
    user = UserRegistry.getUser(cocoon.request.get("name"));
  }
  sendPageAndWait("shippingAddress.xml");
  var address = cocoon.request.get("address");
  sendPageAndWait("creditCard.xml");
  var creditCard = cocoon.request.get("creditCard");
  sendPageAndWait("placeOrder.xml");
  EnterpriseSystem.placeOrder(cart);
  sendPage("orderPlaced.xml");
}
```

sendPageAndWait: a closer look

- `sendPageAndWait("checkout.xml",`
  `{"user": user, "email": email});`

context object

- Context object can be any:
  - Java bean
  - JavaScript object
- Continuation object associated with a unique identifier and available to View

sendPage: send a response

- Tells Cocoon's engine to send out a response and continue the processing
- Usually used before returning from a top-level function
- Could be used to implement simple request/response functions

The View layer

- Flow controller places a dictionary in a context object
- The View accesses only the context object, thus enforcing the separation of concerns

View implementation options

- Multiple options to generate output
  - XSP + JPath logicsheet
  - JPath transformer
  - JXPathTemplate generator
  - Velocity generator

XSP JPath logicsheet

- XSP logicsheet to obtain data from the context object using JXPath
- XPath-like navigation on Java objects
- XSLT-like instructions to retrieve data:
  - `jpath:if`, `jpath:choose`, `jpath:when`, `jpath:otherwise`
  - `jpath:for-each`
  - `jpath:value-of`, `jpath:continuation`

XSP JPath logicsheet (cont.)

- Accesses Java or JavaScript objects from context using XPath expressions
- Retrieve objects from context:

```
<p>First name:
  <jpath:value-of
          select="customer/firstName"/>
</p>
```

XSP JPath logicsheet (cont.)

- Conditionals (`jpath:if`, `jpath:choose`, `jpath:when`, `jpath:otherwise`):

```
<jpath:if test="customer/firstName">
  …
</jpath:if>
```

- Looping (`jpath:for-each`):

```
<jpath:for-each select="customer">
  Name: <jpath:value-of select="name"/>
</jpath:for-each>
```

Sample XSP page

```
<page>
 <s1 title="Login error">
  <jpath:if test="errorMsg">
   <b><jpath:value-of select="errorMsg"/></b>
  </jpath:if>

  <link><xsp:attribute name="href">
          <jpath:continuation/>
        </xsp:attribute>
   Continue
  </link>
</page>
```

JPath transformer

- Transformer instead of an XSP logicsheet
- Supports a language similar to the XSP/JPath logicsheet
- Uses JXPath to access data in the context dictionary

JXPathTemplate generator

- Functionality similar to the JXPath logicsheet, but acts as a Cocoon generator
- Influenced by Velocity, providing a complete set of tags for iteration and conditionals, assigning to variables, defining macros
- Uses either Jexl or JXPath to refer to data in the context object
- Jexl expressions are placed within ${…}
- JXPath expressions are within #{…}

JXPathTemplate

- Conditionals (`if`, `choose`, `when`, `otherwise`):

```
<t:if test="#{customer/firstName}">
  …
</t:if>
```

- Or use `${customer.firstName}`
- Looping (`forEach`):

```
<t:forEach select="#{customer}">
  Name: #{./firstName}
</t:forEach>
```

JXPathTemplate macros

- Provide a simple way to define repetitive tags:

```
<c:macro name="tablerows">
  <c:parameter name="list"/>
  <c:parameter name="color"/>
  <c:forEach var="item" items="${list}">
    <tr><td bgcolor="${color}">${item}</td></tr>
  </c:forEach>
</c:macro>
```

- To use

```
<table>
   <tablerows list="${greatlakes}" color="blue"/>
</table>
```

- In flow script

```
var greatlakes = ["Superior", "Michigan", "Huron", "Erie", "Ontario"];
sendPage(uri, {“greatlakes”: greatlakes});
```

Putting it all together: sitemap

```
<map:flow language="JavaScript">
  <map:script src=”store.js"/>
</map:flow>

<map:match pattern=“checkout”>
  <map:call function=“checkout”
            continuation=“kont/*”/>
</map:match>

<map:match pattern=“checkout.xml”>
  <map:generate src=“checkout.xsp”/>
  …
</map:match>
```

Recap

- Controller is composed of flow scripts written in JavaScript
  - use `sendPageAndWait` to send response to client and temporarily suspend execution
  - use `sendPage` to send response and return immediately
- View could be implemented with:
  - XSP pages using the JXPath logicsheet
  - JPath transformer
  - JXPathTemplate
  - Velocity
- Model: your Java business logic
- Sitemap glues everything together

Questions

Advanced topics

- Global scope of variables
- Tree of continuations
- Expiring continuations
- Handling expired continuations
- Flow object model
- What's next?

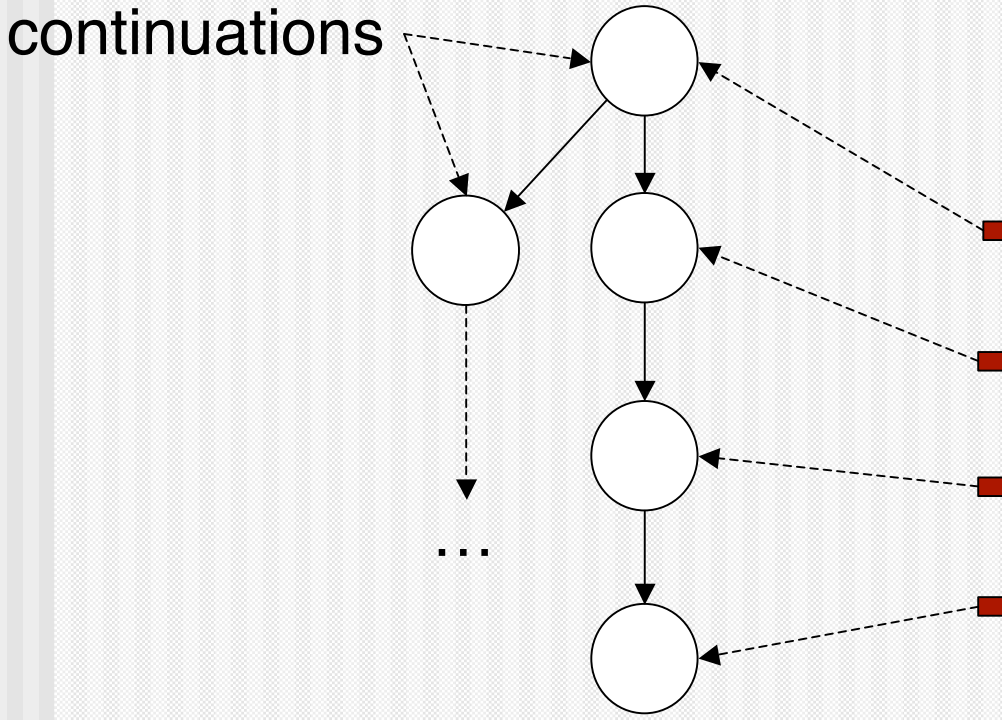
Variables scope

- Each HTTP request creates a new scope for global variables (global scope)
- You can “freeze” or reuse this scope for a user by calling `cocoon.createSession()`
- A continuation also captures the global scope
- This allows the global variables to be saved across top-level function invocations

Variables scope example

```
var user;
function login()
{ user = ……;
  cocoon.createSession();
}
function addItem()
{ // use user here
}
function checkout()
{ // use user here
}
function logout()
{ cocoon.removeSession();
}
```

The browser “back” button




```
var cart;
function checkout()
{
  sendPageAndWait(“login.xml
  var user = UserRegistry.ge
  sendPageAndWait(“shippingA
  var address = request[“add
  sendPageAndWait(“creditCar
  var creditCard = request[“
  sendPageAndWait(“placeOrde
  EnterpriseSystem.placeOrde
  sendPage(“orderPlaced.xml”
}
```

Expiring continuations

- manually expire continuations
  - `sendPageAndWait` returns its continuation k
  - `k.invalidate` invalidates the continuations in its subtree, including itself:

```
var k = sendPageAndWait(…);
……
k.invalidate();
```

- Automatically expire inactive continuations after a period of time

Handling expired continuations

- report an error when a continuation no longer exists

```
<map:handle-errors>
  <map:select type="exception">
     <map:when test="invalid-continuation">
       <map:generate src="documents/invalidContinuation.html"/>
       <map:serialize type="xhtml"/>
     </map:when>
  </map:select>
</map:handle-errors>
```

Flow object model

- Things (objects and functions) available in flow scripts:
  - `cocoon` - global functions
  - `request/response/session/cookie` - HTTP environment
  - `Log` - logging functionality
  - `WebContinuation` - access to the continuation tree

What's next?

- Higher level abstractions on top of the control flow:
  - JXForms
  - Woody

This presenation has ended.
Click anywhere to return.

_

